Digital Hardware Design
Second Lab Assignment Hints

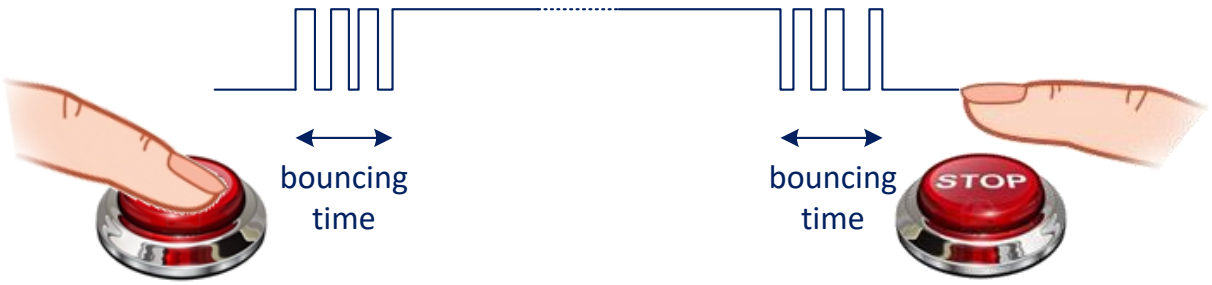
Manuel G. Gericota

Department of Electrical Engineering
School of Engineering – Polytechnic of Porto
&
Fakultet for teknologi og maritime fag
Kongsberg institutt for ingeniørfag
Høgskolen i Sørøst-Norge
Kongsberg, Norway

Debouncing slide switches and push-buttons

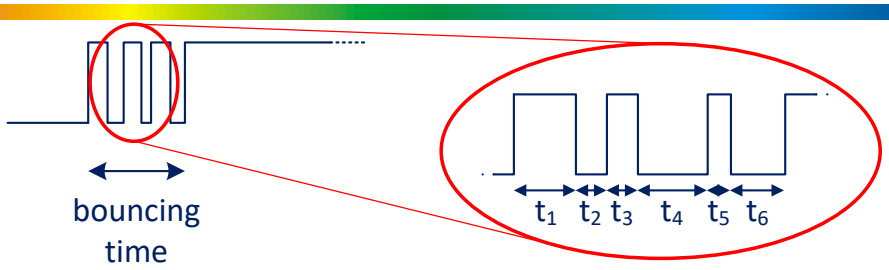
- The debouncing circuit
  - Needed due to the mechanical nature of the button
- When the contacts of a mechanical switch bang together they rebound before settling, causing bounce
- Bouncing $\Rightarrow$ a series of state changes with random intervals much higher than major circuit clock frequency periods
- Debouncing is the process of removing the bounces

Debouncing slide switches and push-buttons



- Bouncing time < 10 ms typically ⇒ much higher than the clock period
- Switch output reaches stability only after the bouncing time

Debouncing slide switches and push-buttons



- In the meanwhile, a series of output switch state changes occur
- t_1, t_2, t_3, t_4, t_5, t_6 > T (period of the clock frequency) ⇒ the circuit sees a series of switch changes and not a single clean switch change as intended

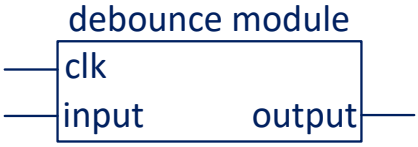
Debouncing slide switches and push-buttons

- Using the RT methodology, a Finite State Machine may be used to control a timer to obtain a precise time interval after bouncing's end
- Debounced switching occurs only after a certain delay time after the last bouncing transition
- Delay time ⇒ assures the switch has already achieved stability

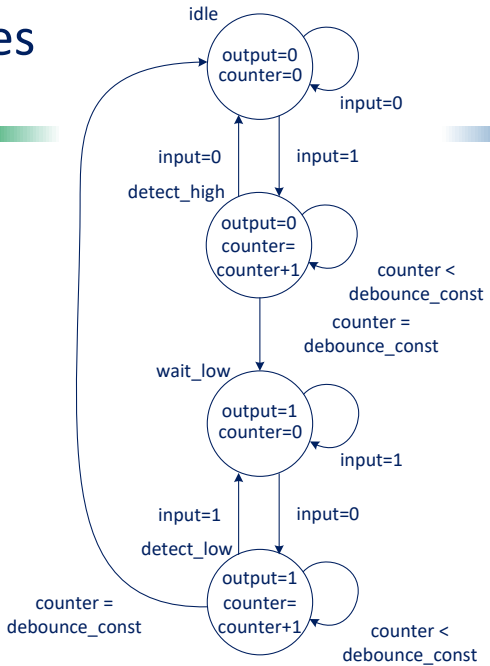


Debouncing slide switches and push-buttons

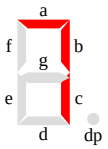
- FSM diagram for the debouncing circuit



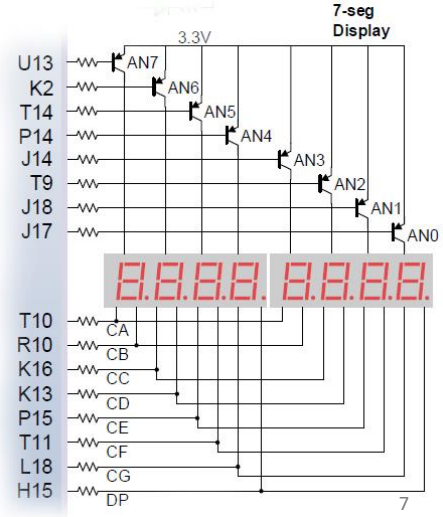
- For a delay of 10 ms with a clock frequency of 100MHz ⇒ 20-bit counter is needed



7-segment display decoder

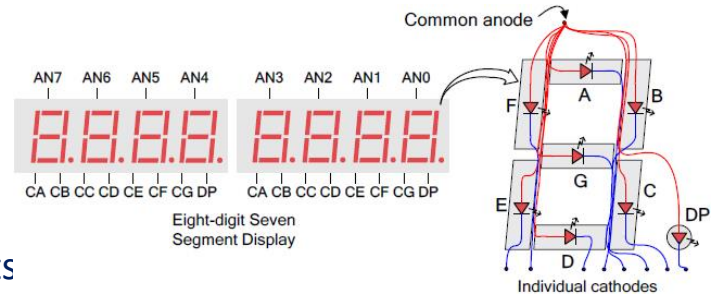
- Hex-to-7-segment decoder
  - ex: 7 → 

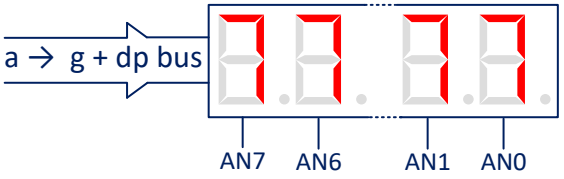
→ 1 0 0 0 1 1 1 1
dp a b c d e f g
  - to light a segment ⇒ signal <= 0 ⇒ in the example, “10001111”
  - 16 possibilities → 0 to 9, plus ‘a’, ‘b’, ‘c’, ‘d’, ‘e’, ‘f’



7-segment display decoder

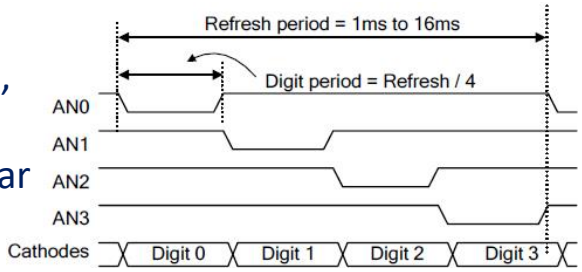
- Example for a 8 7-segment display
- Each digit **shares** the same eight common control signals to light individual LED segments
- Each individual character has a separate anode control input





7-segment display decoder

- LED control signals ⟶ time-multiplexed to display data on all eight characters
  - present the value to be displayed on the segment control inputs
  - select the specified character by driving the associated anode control signal Low
- Through persistence of vision, the human brain perceives that all eight characters appear simultaneously



7-segment display decoder implementation

- Each of the two blocks should be implemented separately
  - hex-to-7seg module
  - disp_mux module
- In this way, you will be able to reuse each block if need in the next designs

Remember the rules and steps stated in the Introduction part

- First
  - understand the problem and divide it in parts (blocks)

Understand and use standard functional building blocks



Binary-to-BCD conversion algorithm

- Conversion Algorithm
  - Double-Dabble Binary-to-BCD Conversion Algorithm *
- Algorithm description
  - if the number to be converted is stored in a n-bit register → create a register with a length equal to n-bit + the number of bits needed to store its representation in BCD

* Rhyne, V.T., "Serial Binary-to-Decimal and Decimal-to-Binary Conversion," *IEEE Transactions on Computers*, Vol. C-19, No. 9, pp. 808-812, Sept. 1970

Binary-to-BCD conversion algorithm

- store the original value in the least significant bits of the new register and initialize the new bits to zero
- divide the extra space in BCD digits (4-bits each digit) starting from the MSB

  ex: 0000 0000 0000 10110010
- left-shift the entire register space one bit
- however, *before* the left-shift is done, any BCD digit which is greater than 4 is incremented by 3 (the increment ensures that a value of 5, incremented and left-shifted, becomes 16, thus correctly "carrying" into the next BCD digit)

Binary-to-BCD conversion algorithm

- repeat the shifting many times as the number of bits of the original number

ex: 0000 0000 1011 ⟶ begin – 4 bits (11_{10})
0000 0101 1xxx ⟶ after 3 shifts the 1st BCD is greater than 4
0000 1000 1xxx ⟶ add 3 to the 1st BCD digit
0001 0001 xxxx ⟶ after the 4th and last shift – 11BCD ⇒ 11_{10}

Binary-to-BCD conversion algorithm

Another example

1. 8-bit initial value 10110110_2 → 182_{10}
2. add enough BCD digits to the left and apply the algorithm
3. 0000 0000 0000 10110110 → begin – 8 bits (182_{10}) ⇒3 BCD digits
4. 0000 0000 0001 0110110x → 1st shift
5. 0000 0000 0010 110110xx → 2nd shift
6. 0000 0000 0101 10110xxx → after the 3rd shift the 1st BCD > 4
7. 0000 0000 1000 10110xxx → result of adding 3 to 1st BCD digit
8. 0000 0001 0001 0110xxxx → 4th shift
9. 0000 0010 0010 110xxxxx → 5th shift
10. 0000 0100 0101 10xxxxxx → after the 6th shift the 1st BCD > 4
11. 0000 0100 1000 10xxxxxx → result of adding 3 to 1st BCD digit
12. 0000 1001 0001 0xxxxxxx → after the 7th shift the 2nd BCD > 4
13. 0000 1100 0001 0xxxxxxx → result of adding 3 to 2nd BCD digit
14. 0001 1000 0010 xxxxxxxx → 8th and last shift (*n* = 8 bits)